



Multi-agent Adversarial Search

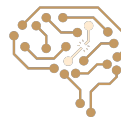
Arash Marioriyad

13 Ordibehesht 1405

CE 417-Artificial Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188





Up to Now ...

- **Uninformed Search:** BFS, DFS, IDS, UCS, ...
- **Informed Search:** Greedy, A^* , Heuristics, ...
- **Local Search:** Hill Climbing, SA, Genetic Algorithm, ...
- **CSP:** Backtracking, Forward Checking, Arc Consistency, ...

- The common factor: **Single Agent** Scenarios

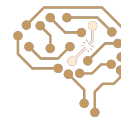
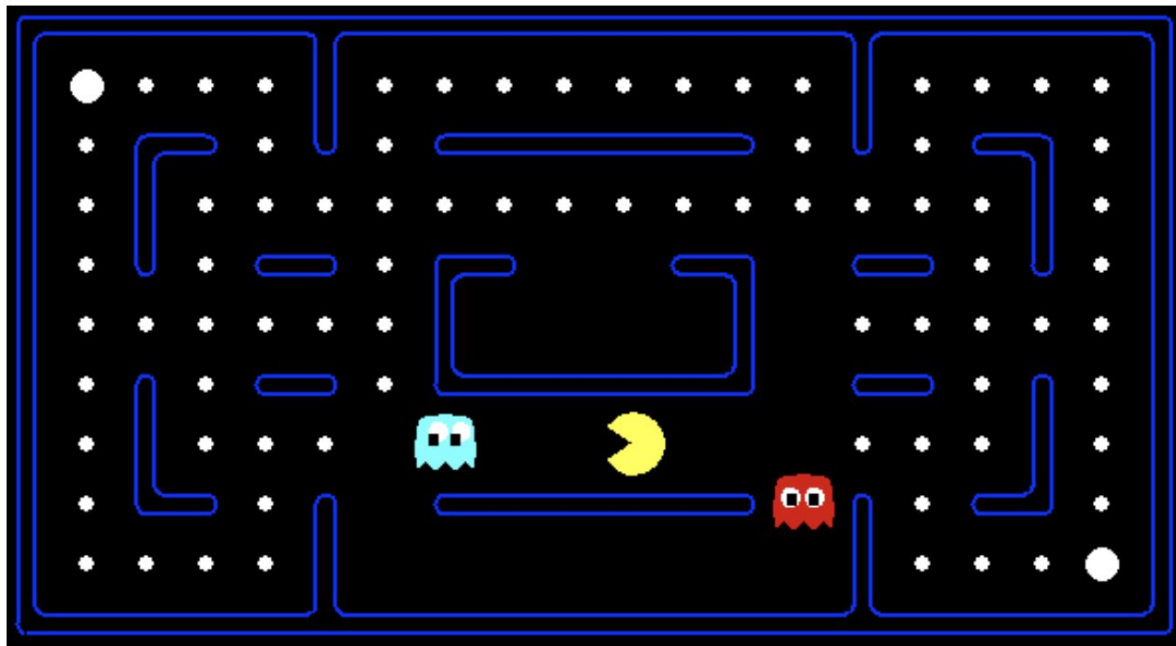


Today ...

- Multi Agent / Adversarial / Zero-sum Games
- Minimax Search
- Pruning
- Evaluation
- Uncertainty

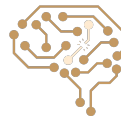


Multi Agent Search Problem



Why Multiple Agents?

- Playing multi-agent Games! (e.g., chess)
- AI that operates alongside humans or other AIs
- Multiple agents lead to more complex environments/ecosystems
 - Inspired by evolution
 - Games, robotics, generative adversarial networks (GANs)
- We'll focus on games, but multi-agent ideas come up in many areas of AI



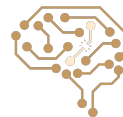
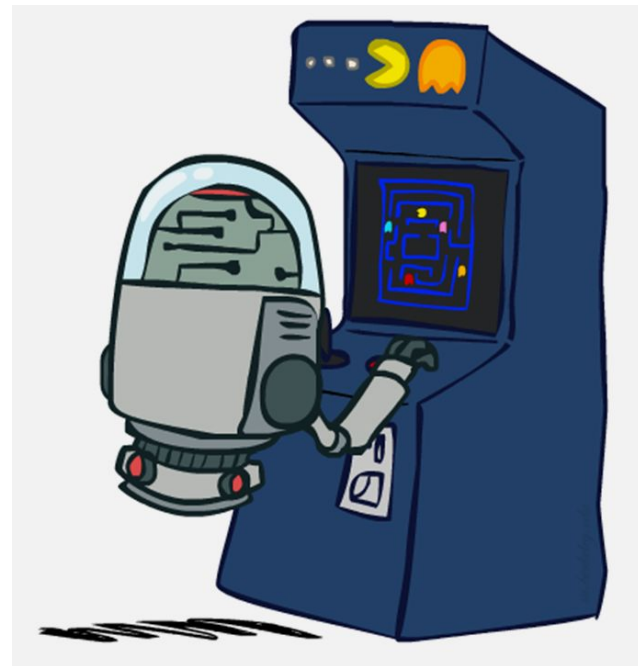
Characteristics of Games

- Game = task environment with > 1 agent
- Deterministic or stochastic?
- Perfect information (fully observable)?
- Two, three, or more players?
- Teams or individuals?
- Turn-taking or simultaneous?
- Zero sum?

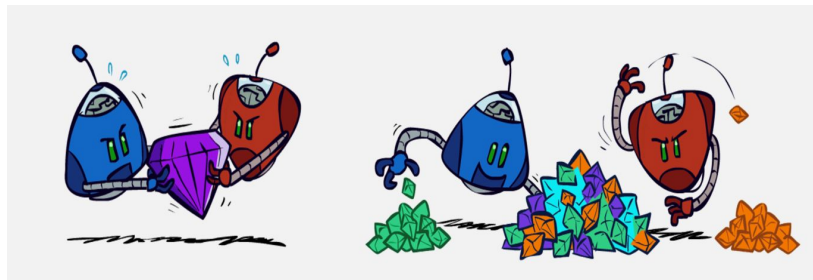


Deterministic Games Formulation

- **States:** S (start at s_0)
- **Players:** $P = \{1, \dots, N\}$ (usually take turns)
- **Actions:** A (may depend on player / state)
- **Transition function:** $S \times A \rightarrow S$
- **Terminal test:** $S \rightarrow \{T, F\}$
- **Terminal utilities:** $S \times P \rightarrow R$
- Solution for a player is a **Policy:** $S \rightarrow A$



Types of Games



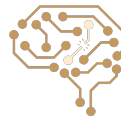
Zero-Sum Games

- Agents have **opposite** utilities
- Lets us think of a single value that one maximizes and the other minimizes

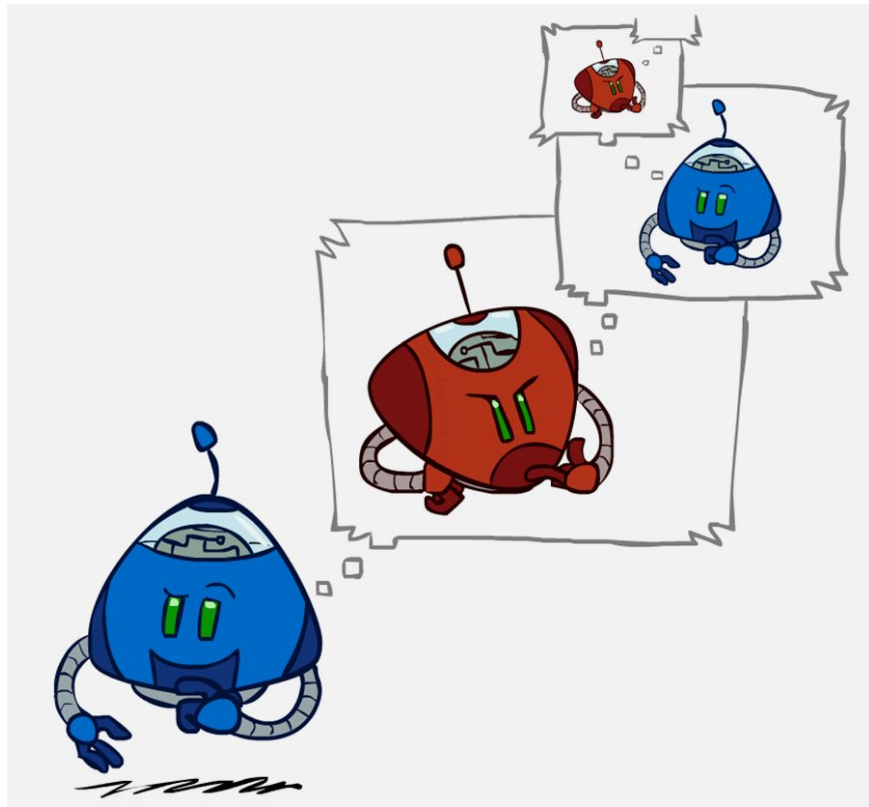
Adversarial, pure competition

General Games

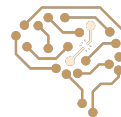
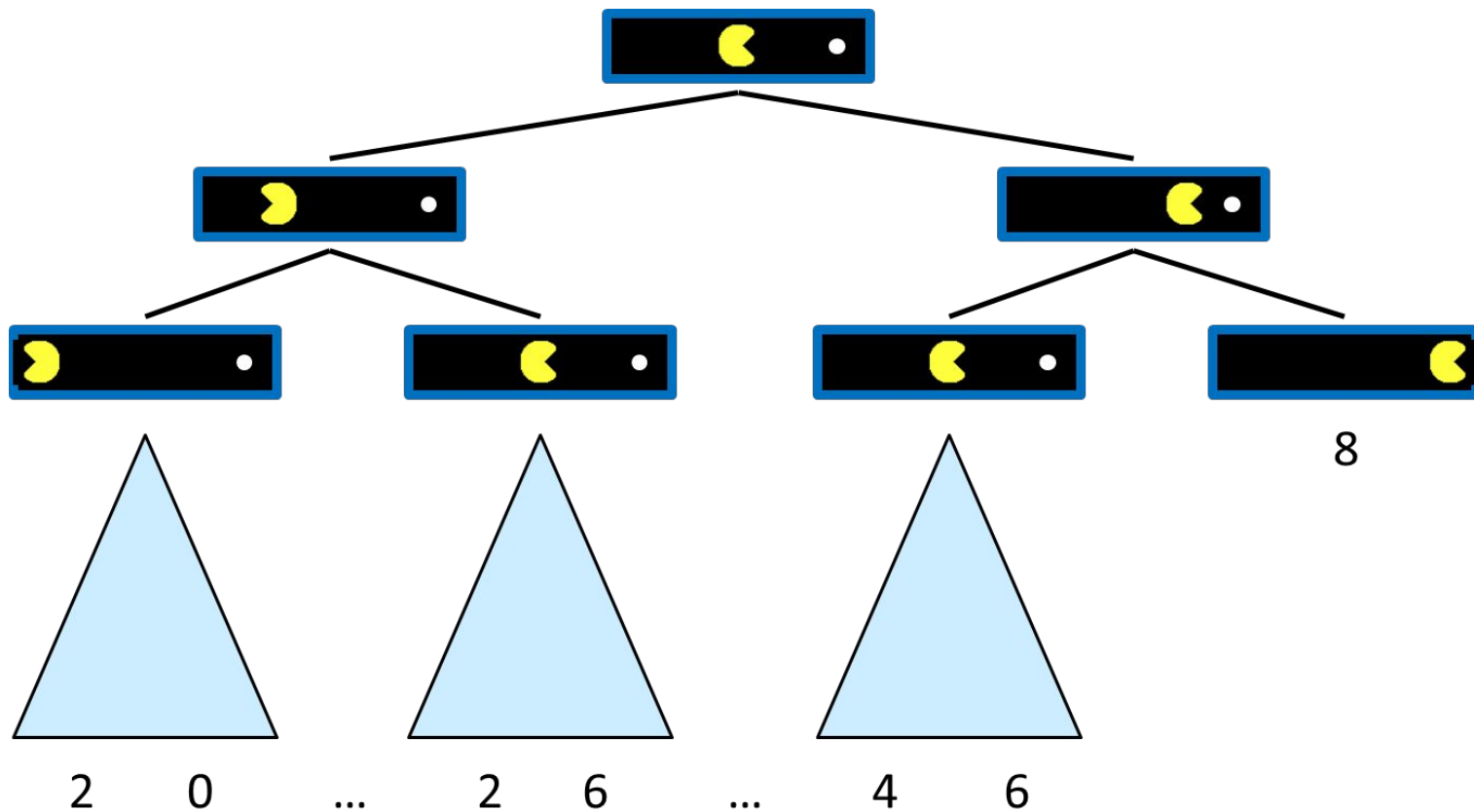
- Agents have **independent** utilities
- Cooperation, indifference, competition, & more are possible



Adversarial Search

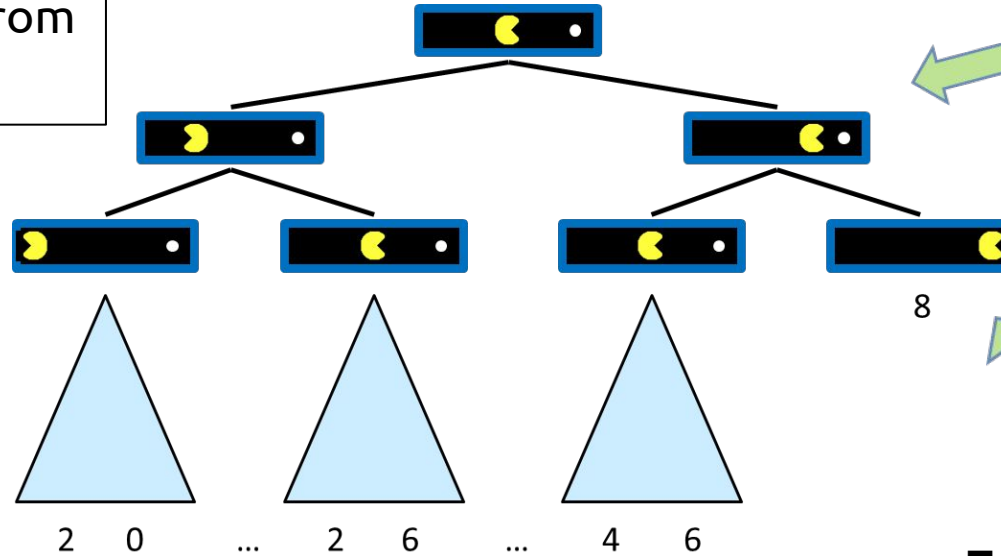


Single-Agent Trees



Value of a State

The best achievable outcome (utility) from that state



Non-Terminal States:

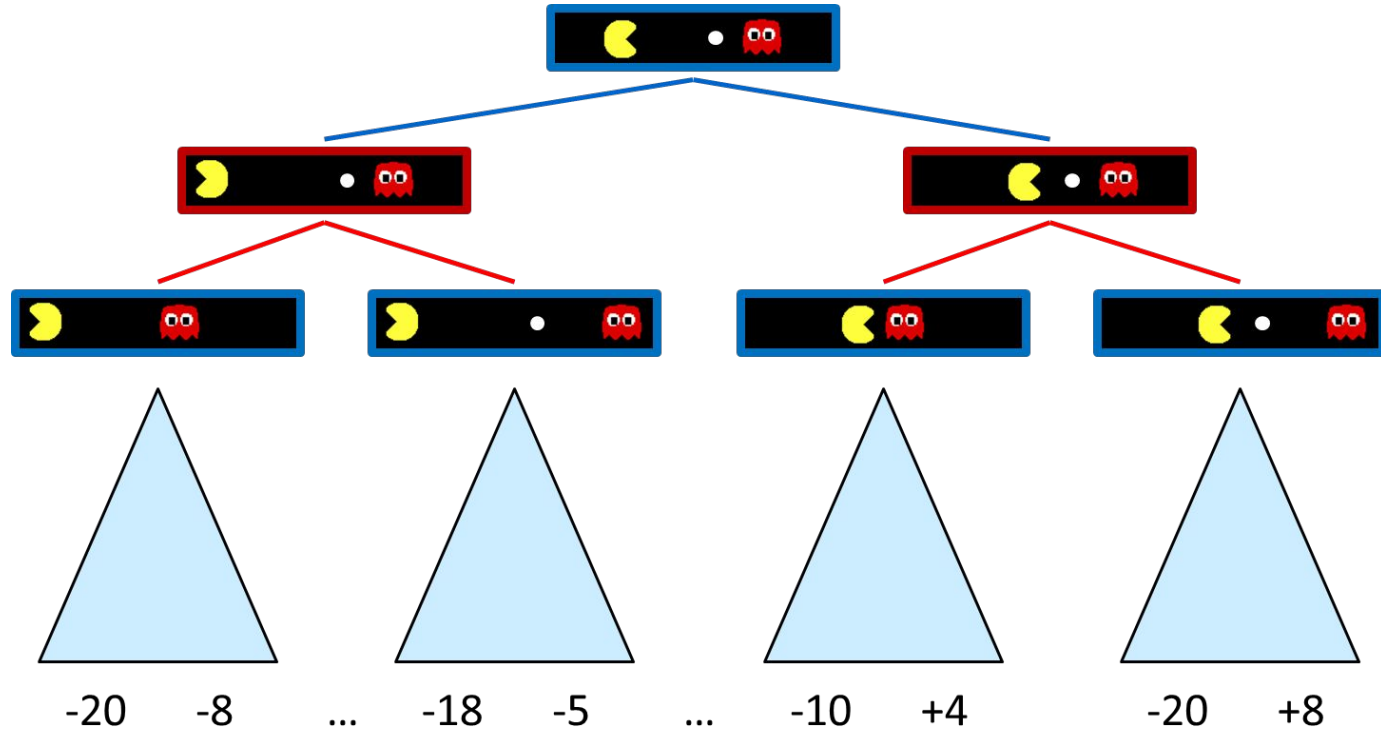
$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

Terminal States:

$$V(s) = \text{known}$$



Adversarial Game Trees



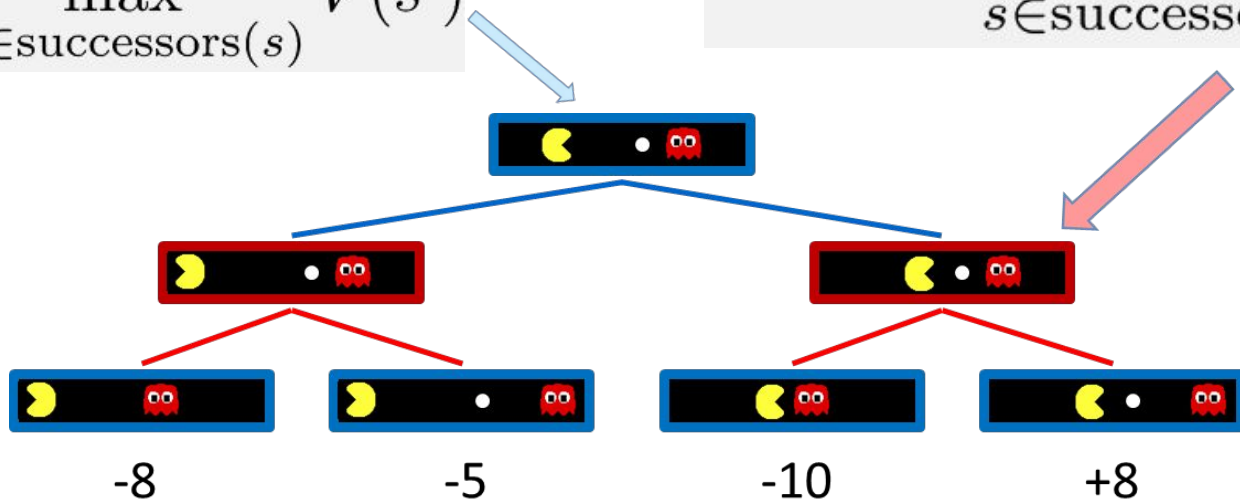
Minimax Values

States Under Agent's Control:

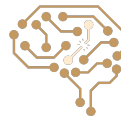
$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

States Under Opponent's Control:

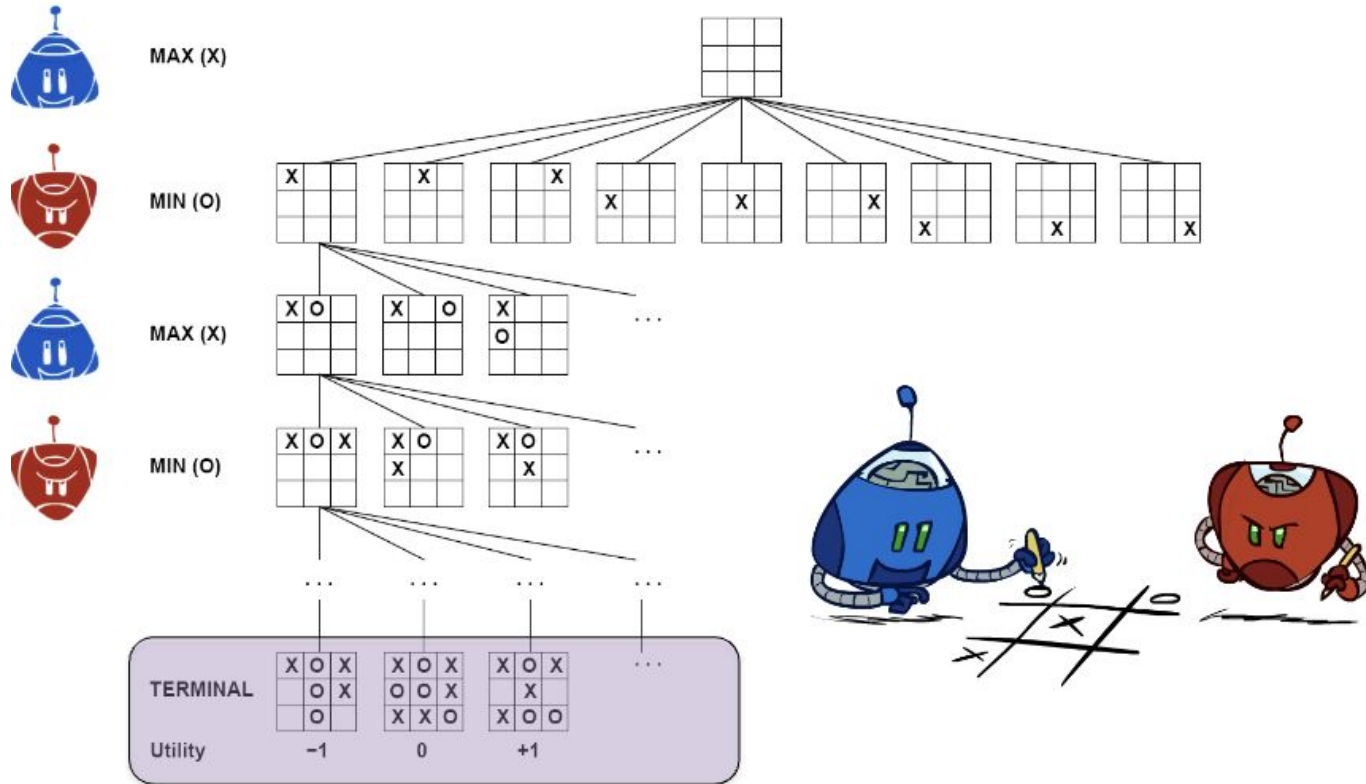
$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$



Terminal States:
 $V(s) = \text{known}$



Tic-Tac-Toe Game Tree



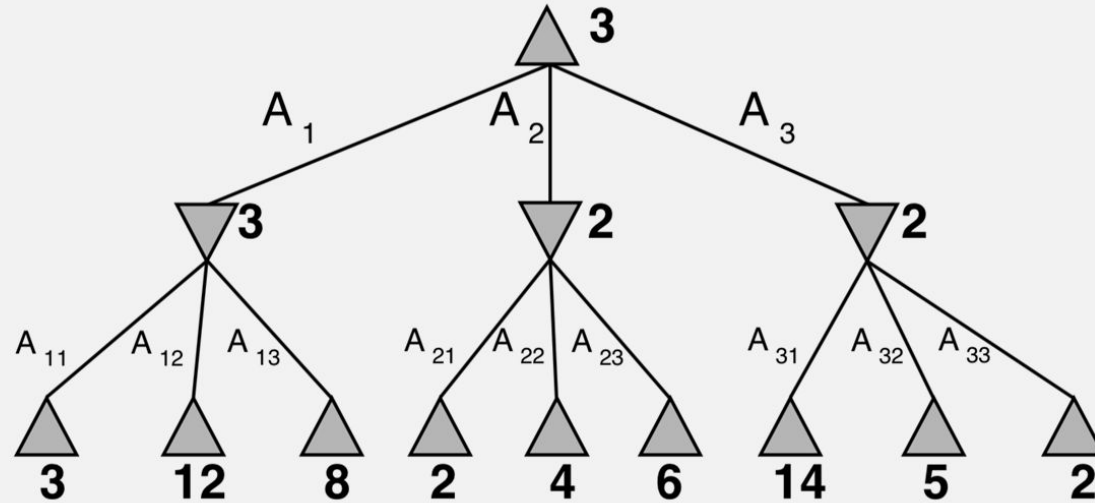
Minimax Strategy

- Perfect play for deterministic, perfect-information games
- Idea: best achievable payoff against best play

E.g., 2-ply game:

MAX

MIN



Minimax Implementation (DFS-like)

```
def value(state):
```

if the state is a terminal state: return the state's utility

if the next agent is MAX: return max-value(state)

if the next agent is MIN: return min-value(state)

```
def max-value(state):
```

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

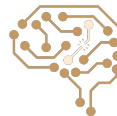
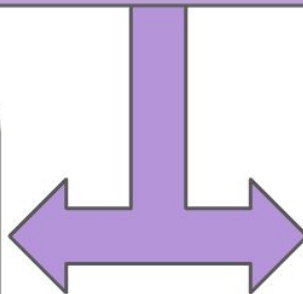
```
def min-value(state):
```

initialize $v = +\infty$

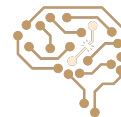
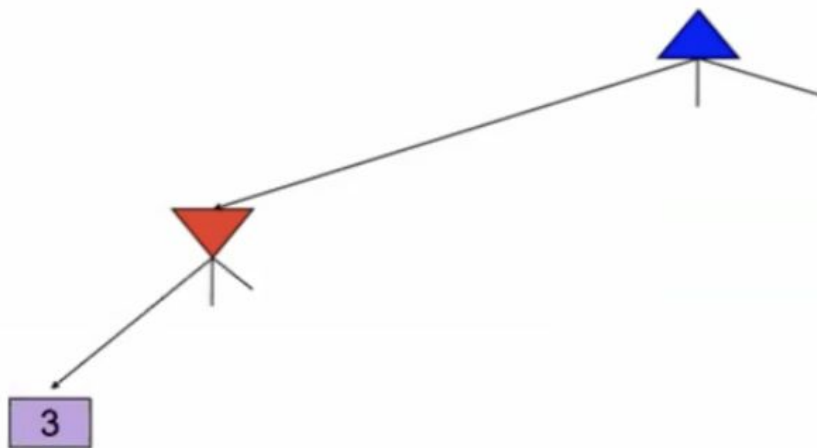
for each successor of state:

$v = \min(v, \text{value}(\text{successor}))$

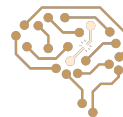
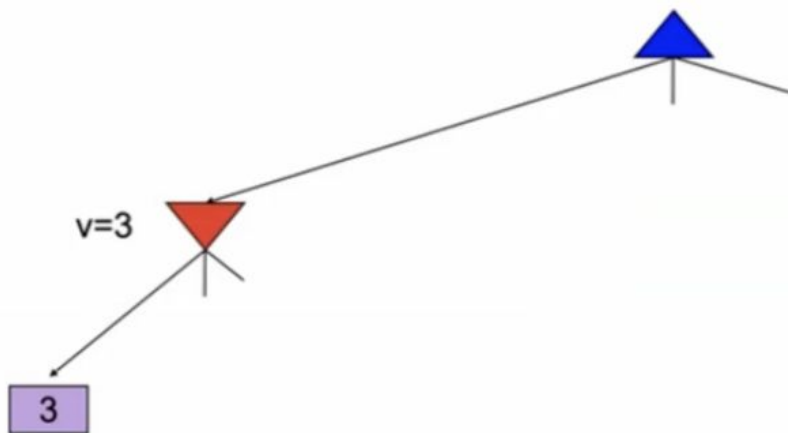
return v



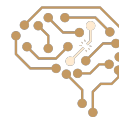
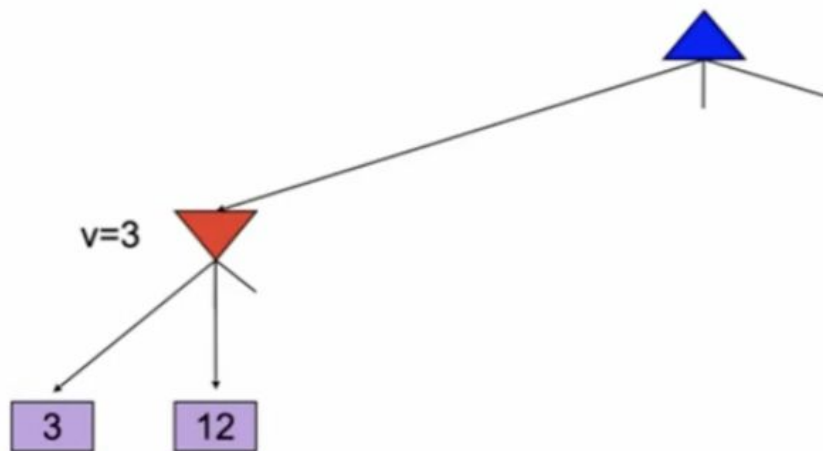
Minimax Example



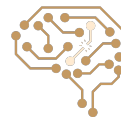
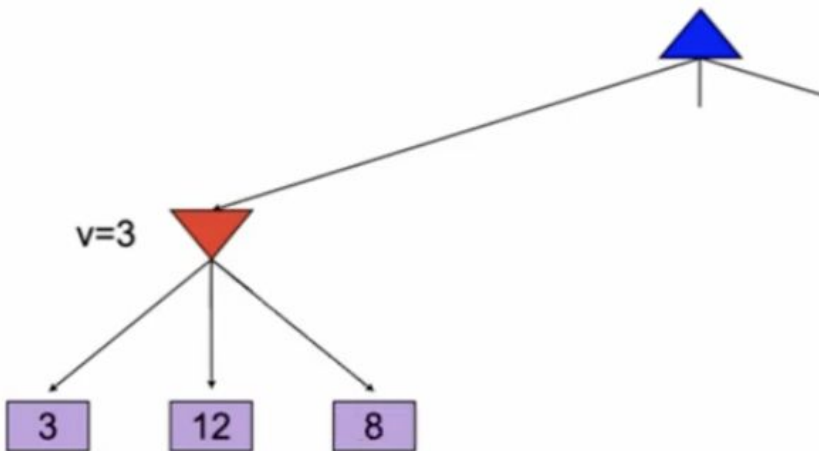
Minimax Example



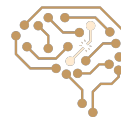
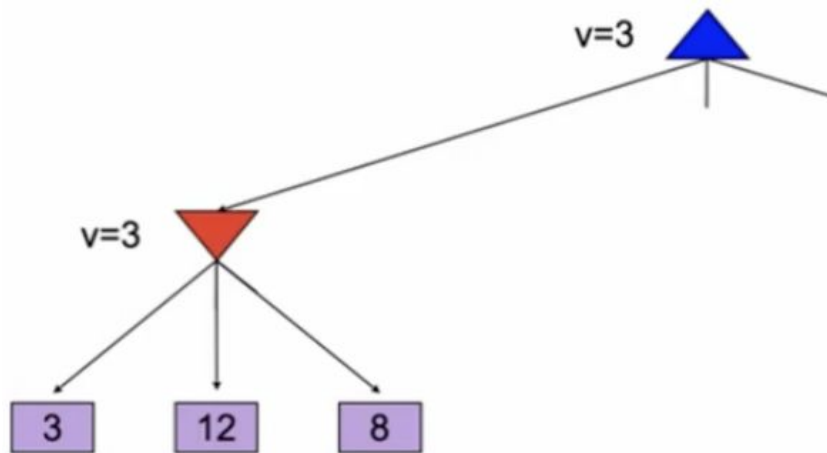
Minimax Example



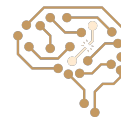
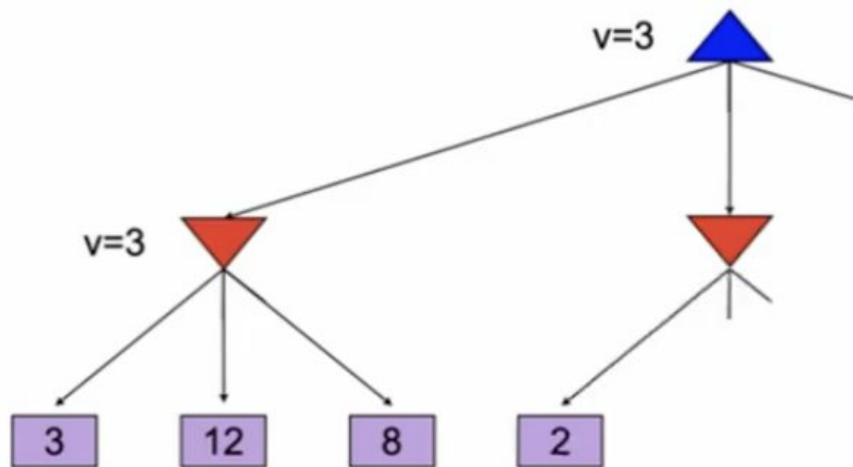
Minimax Example



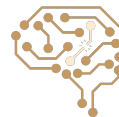
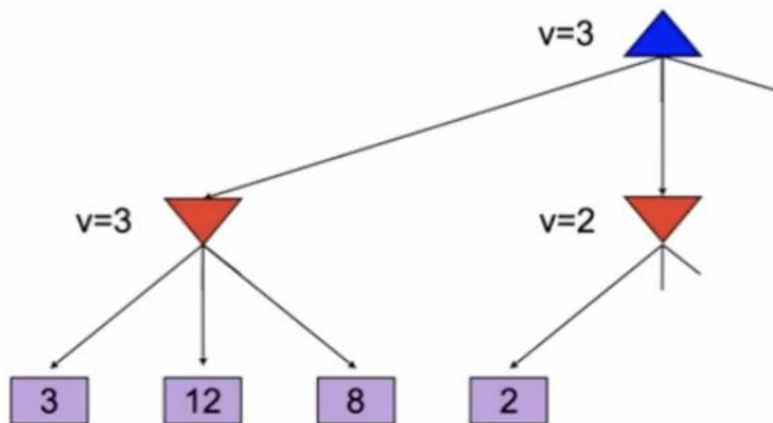
Minimax Example



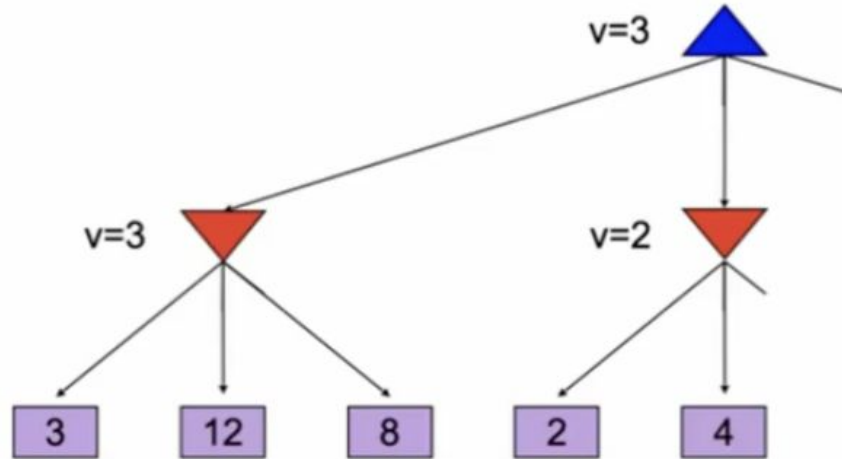
Minimax Example



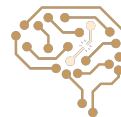
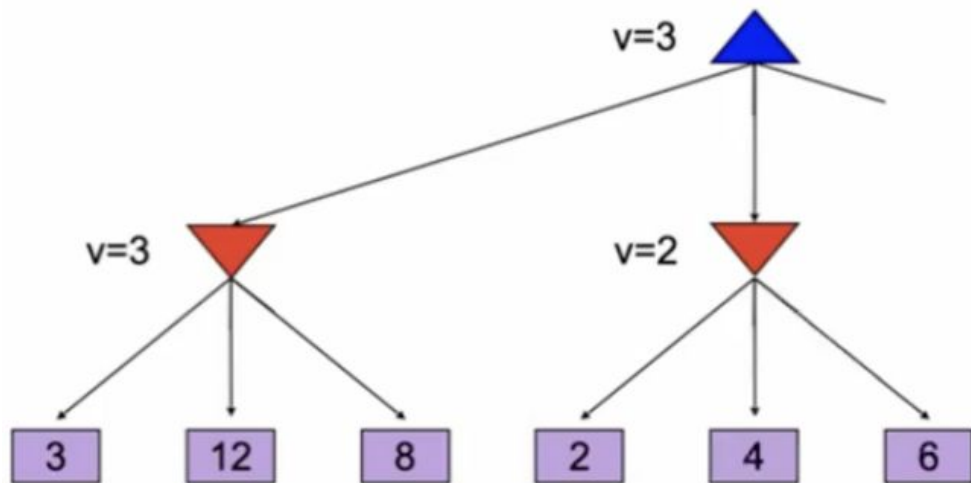
Minimax Example



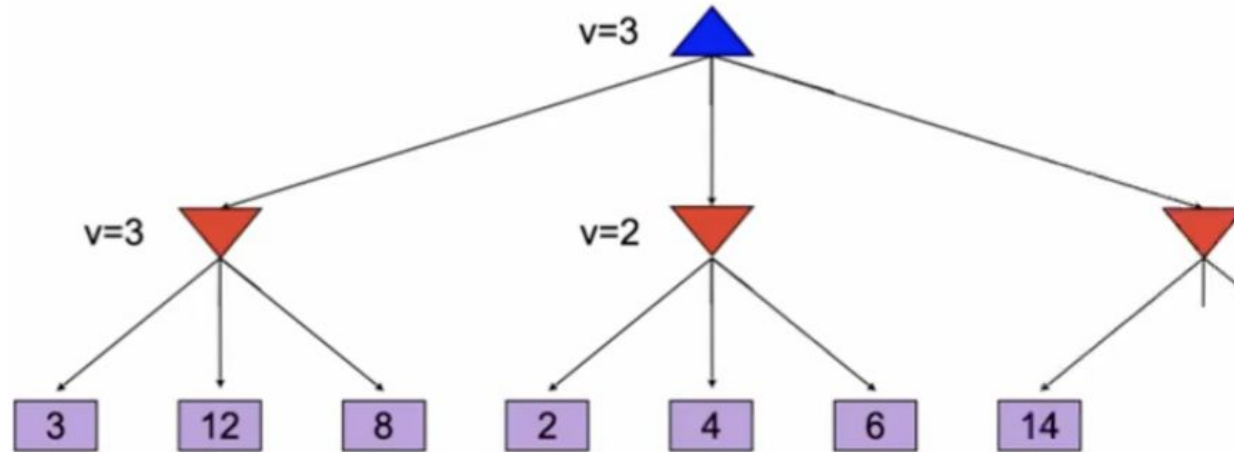
Minimax Example



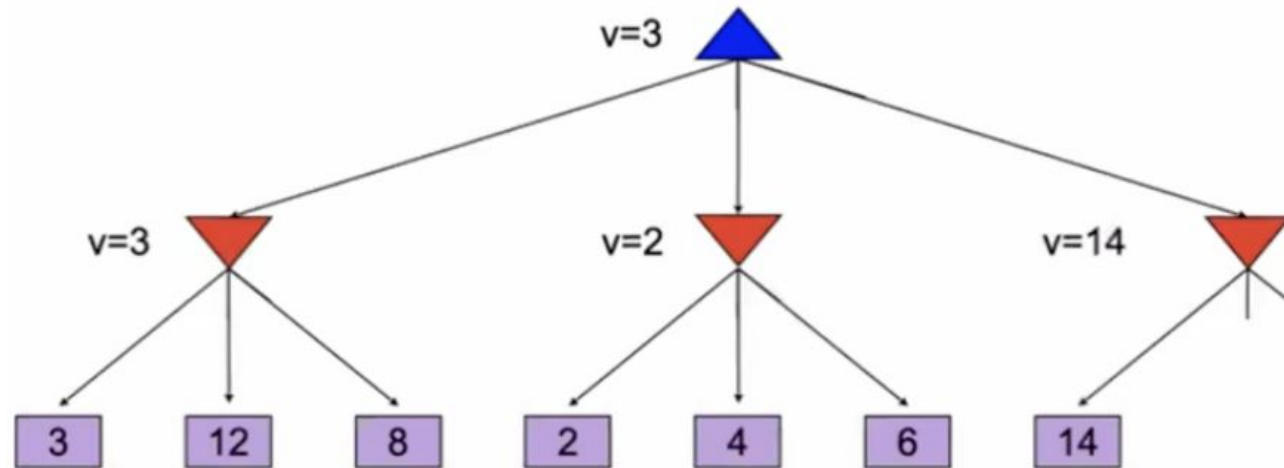
Minimax Example



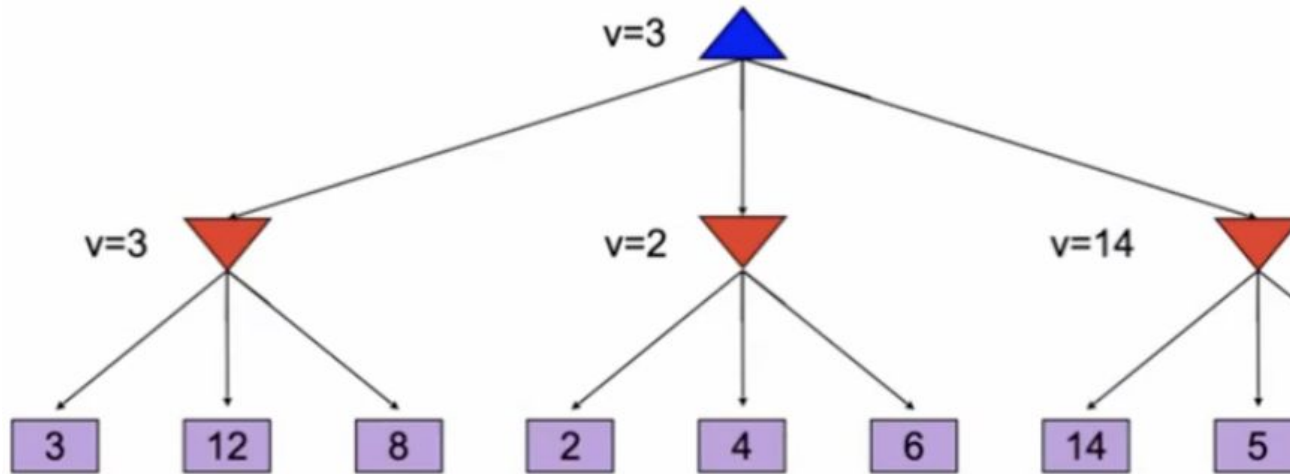
Minimax Example



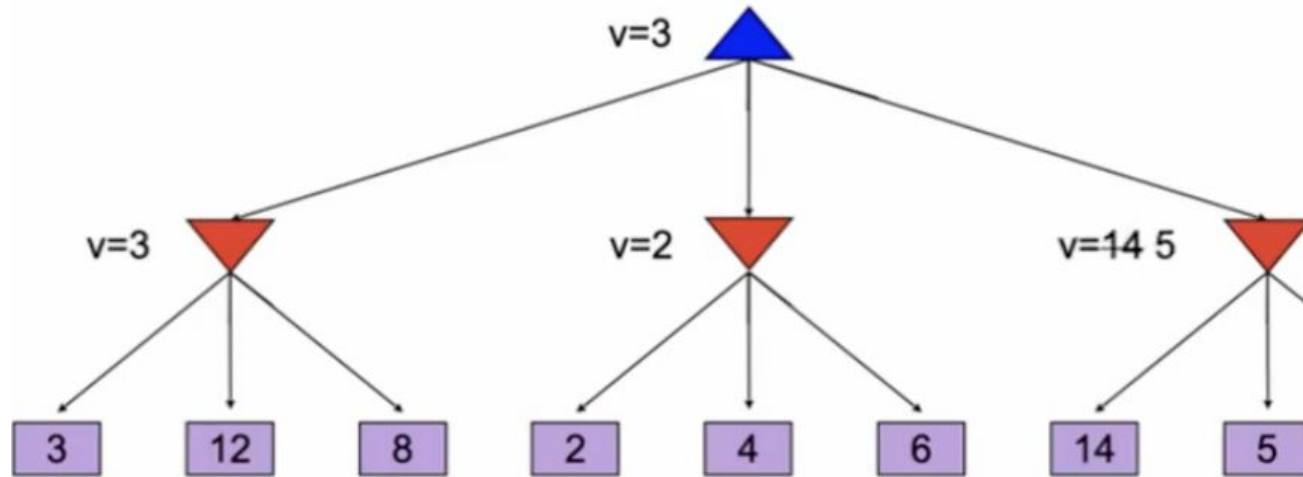
Minimax Example



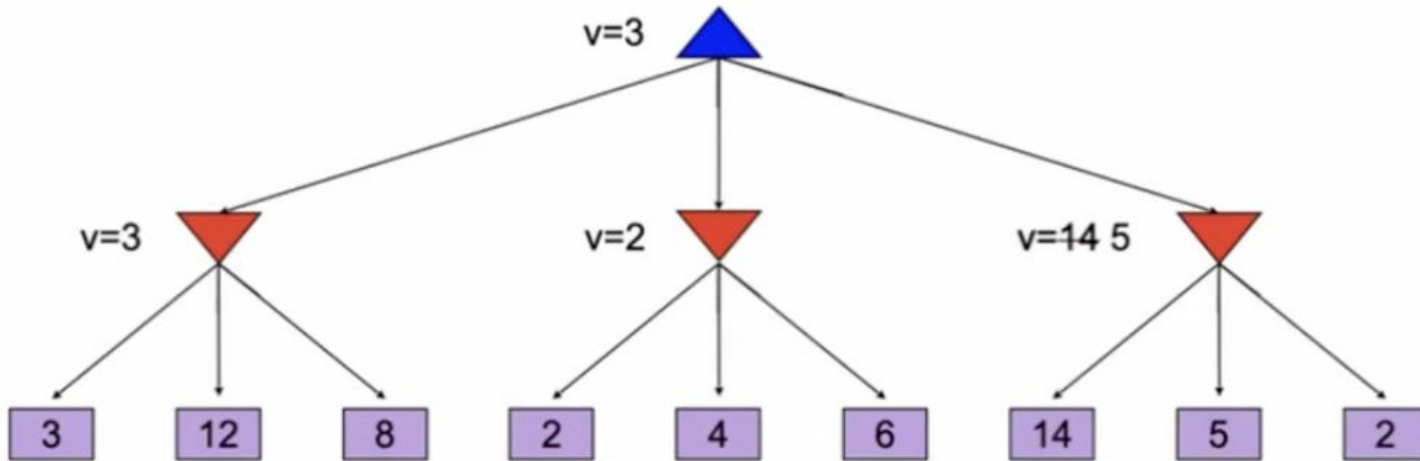
Minimax Example



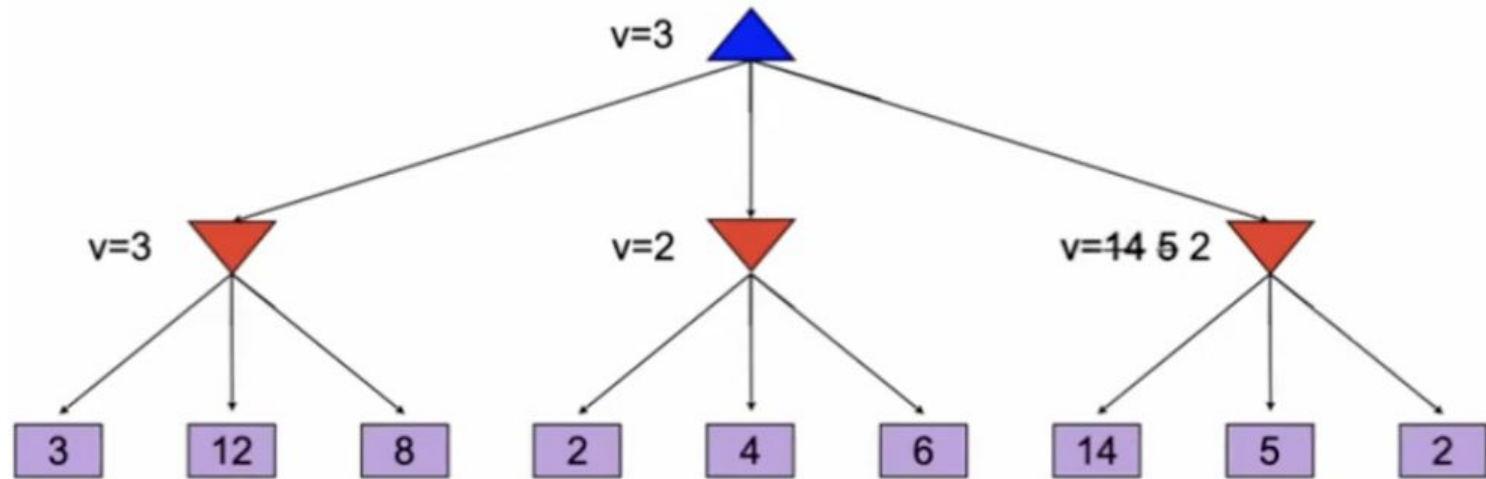
Minimax Example



Minimax Example

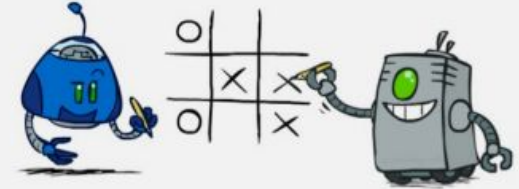
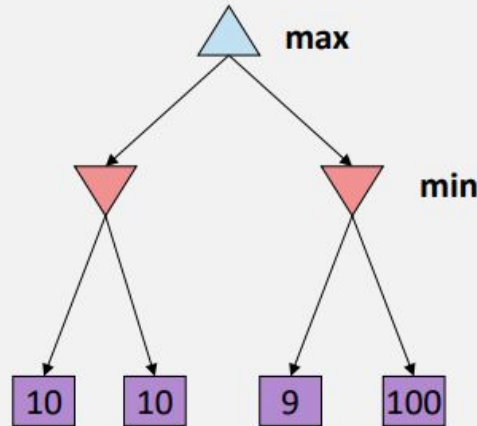
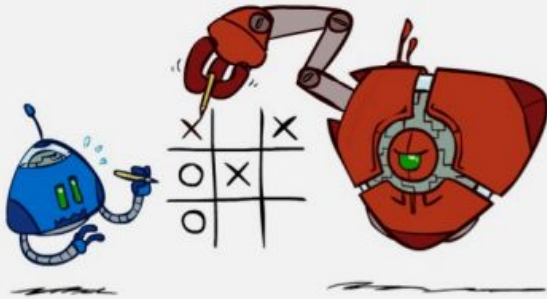


Minimax Example



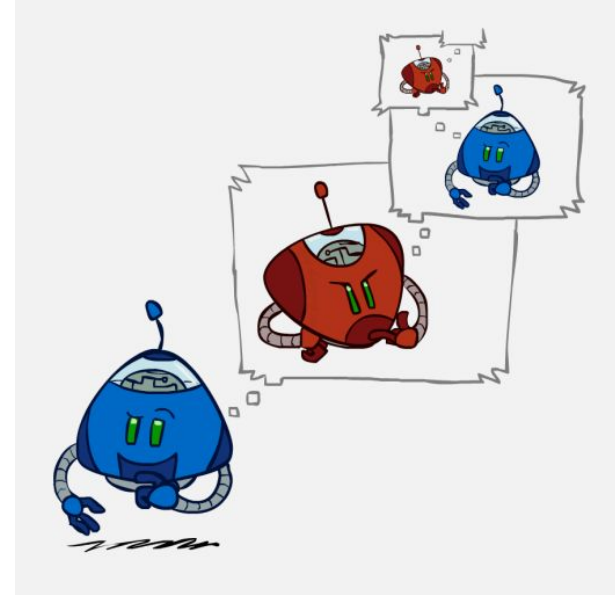
Minimax Properties

- Optimal against a perfect player. Otherwise?

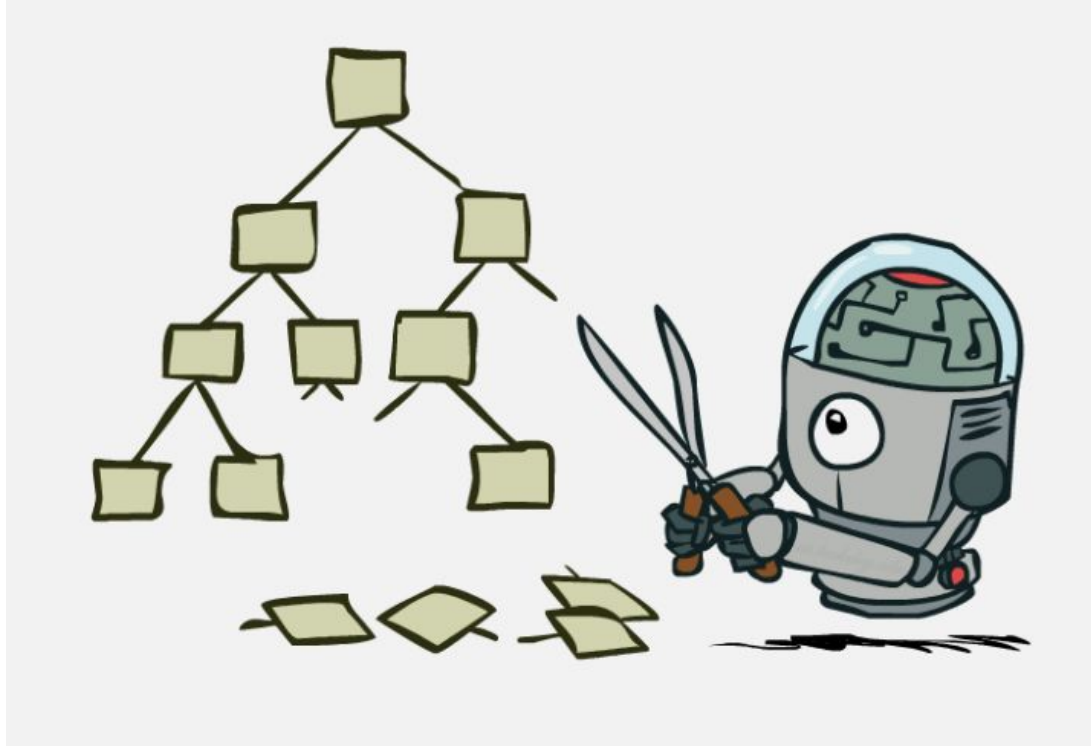


Minimax Efficiency

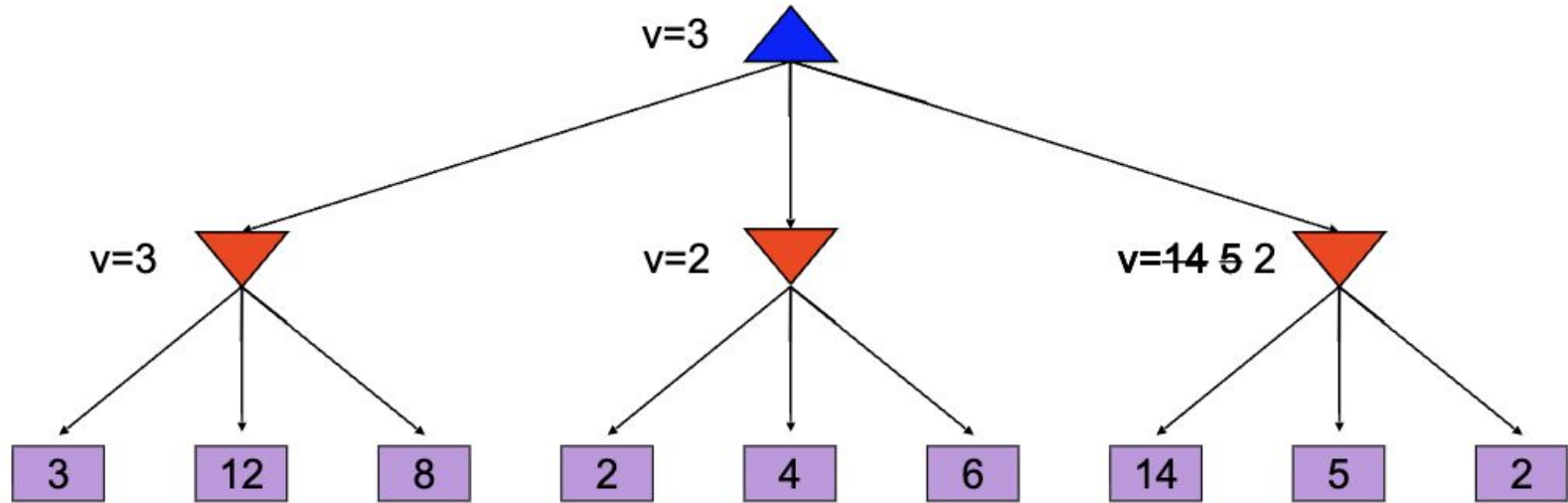
- How efficient is minimax?
 - Just like (exhaustive) DFS
 - Time: $O(b^m)$
 - Space: $O(bm)$
- Example: For chess, $b \approx 35$, $m \approx 100$, For Go, $b \approx 250-300$, $m \approx 150$
 - Exact solution is completely infeasible
 - But, do we need to explore the whole tree?



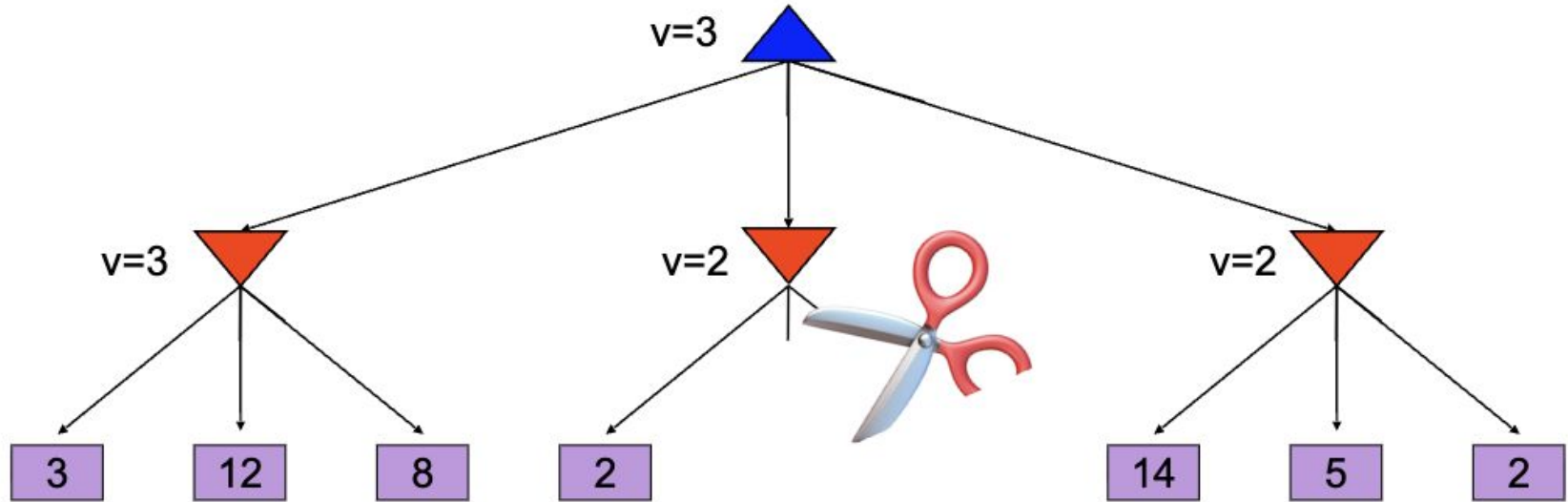
Game Tree Pruning



Minimax Pruning Example



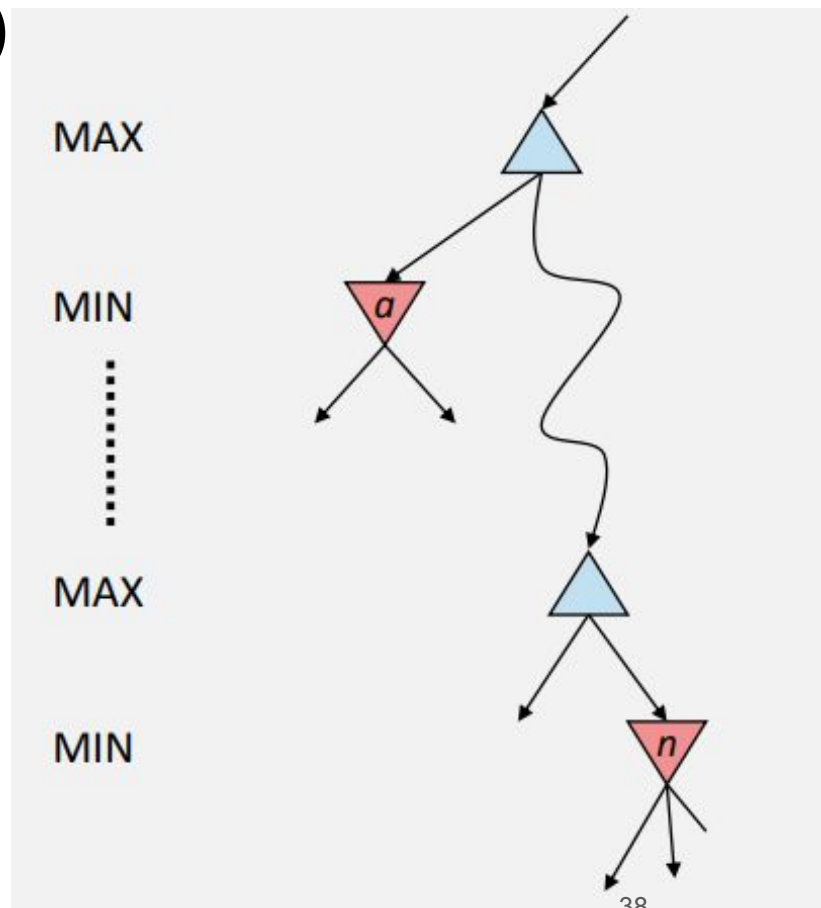
Minimax Pruning Example



Alpha-Beta Pruning (Min Version)

- We're computing the MIN-VALUE at some node n
- We're looping over n 's children
- Who cares about n 's value? MAX
- Let α be the best value that MAX can get at any choice point along the current path from the root
- **If n becomes worse than α , MAX will avoid it**
- So we can stop considering n 's other children
- it's already bad enough that it won't be played

MAX version is symmetric



Alpha-Beta Pruning Implementation

α : MAX's best option on path to root so far
 β : MIN's best option on path to root so far

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \geq \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \leq \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

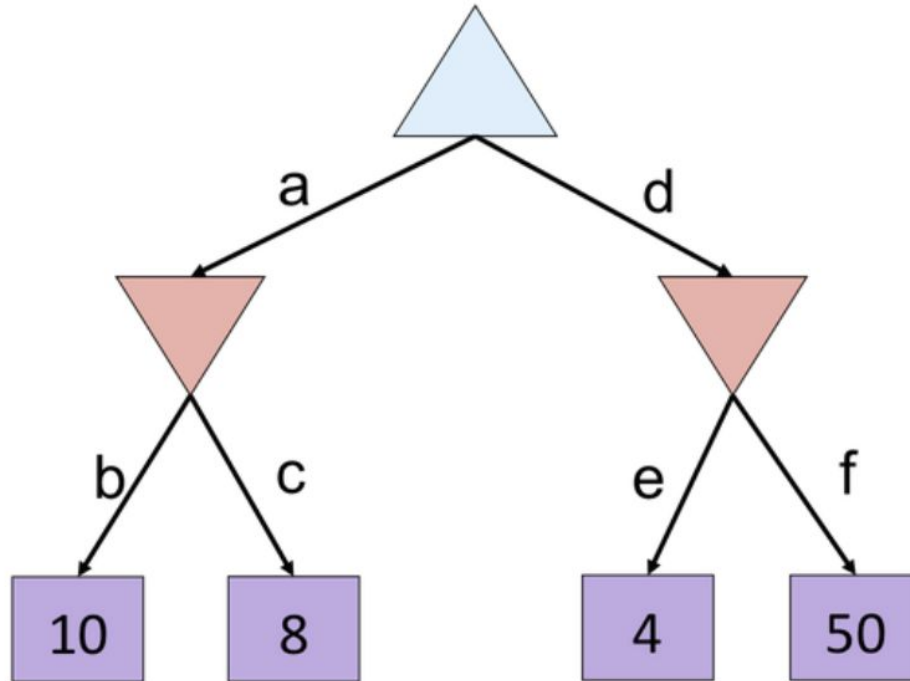


Alpha-Beta Pruning Properties

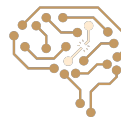
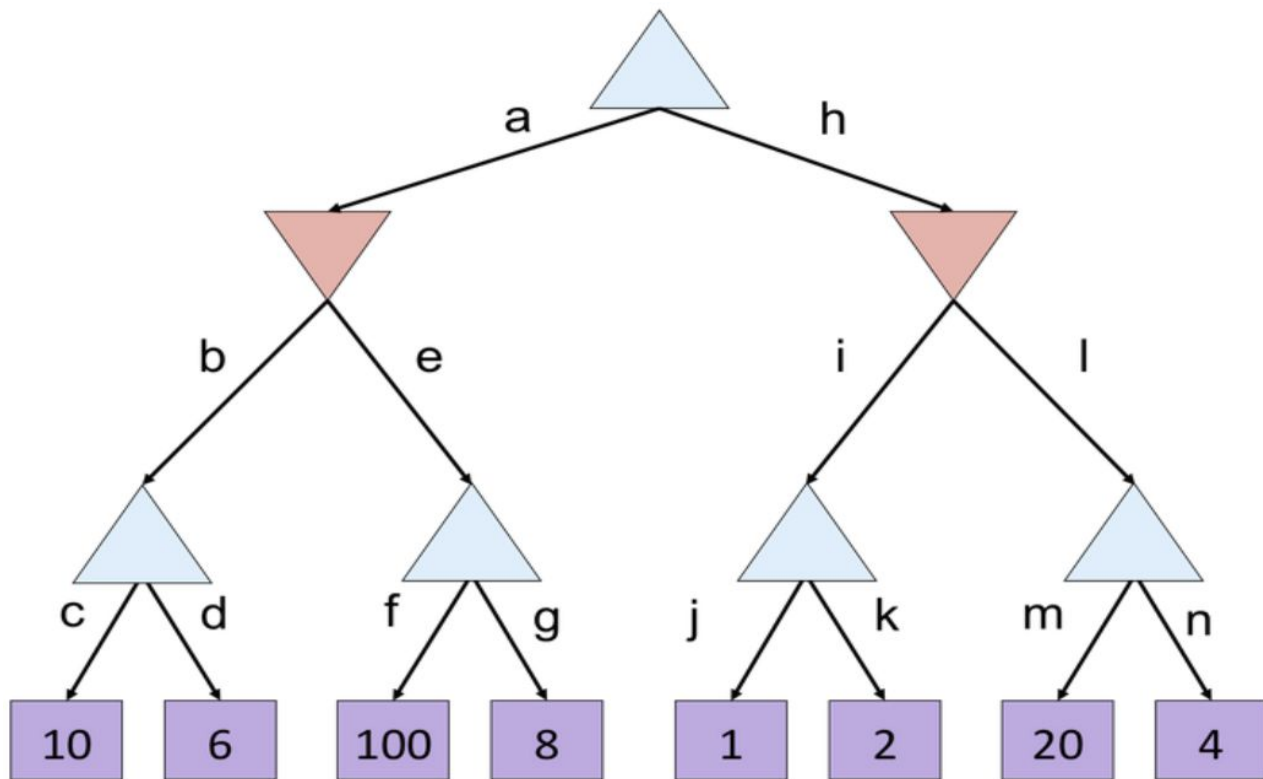
- The pruning has no effect on minimax value computed for the root!
- Values of intermediate nodes might be wrong
- Good child ordering improves effectiveness of pruning
- With “perfect ordering”: Time complexity drops to $O(b^{m/2})$



Alpha-Beta Pruning Quiz 1



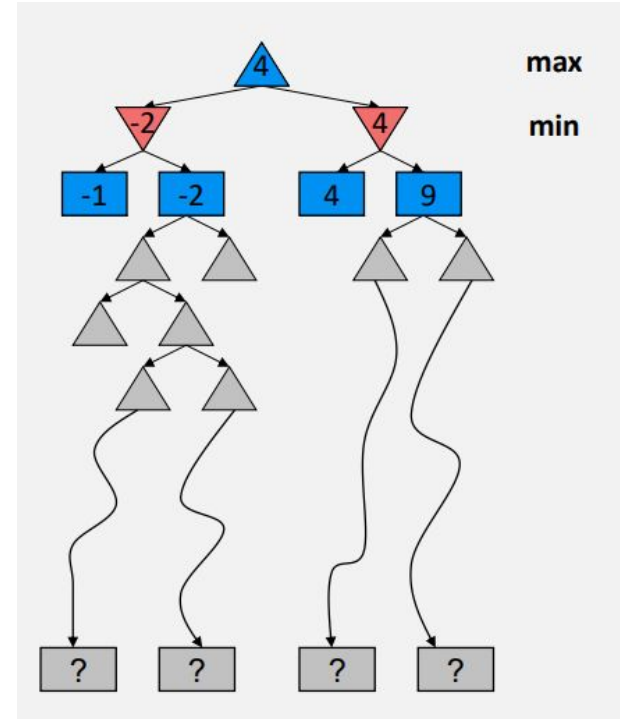
Alpha-Beta Pruning Quiz 2



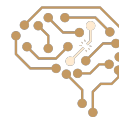
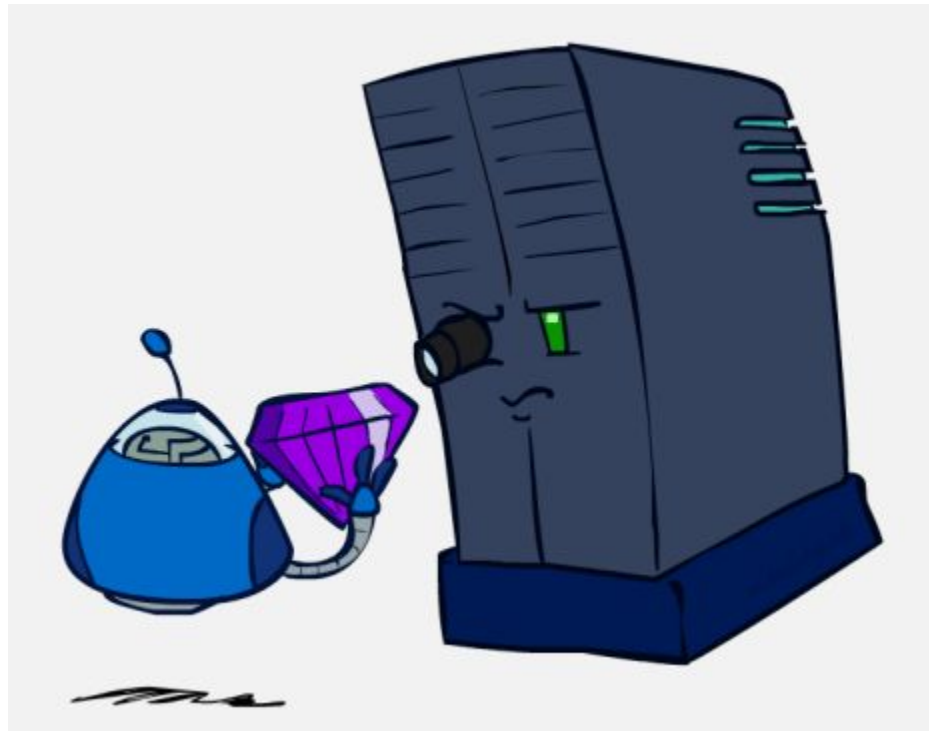
Limiting Depth

- Problem: In realistic games (like chess), cannot search to leaves!
- Solution: Depth-limited search
 - Instead, search only to a limited depth in the tree
 - Replace terminal utilities with an **evaluation function** for non-terminal positions
- Guarantee of optimal play is gone

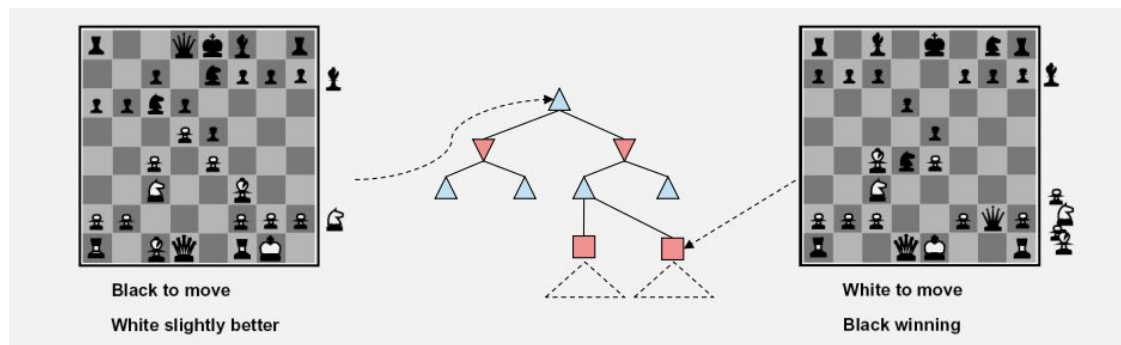
Use iterative deepening for an anytime algorithm



Evaluation Functions



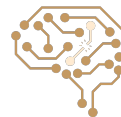
Evaluation Functions



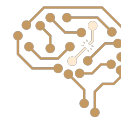
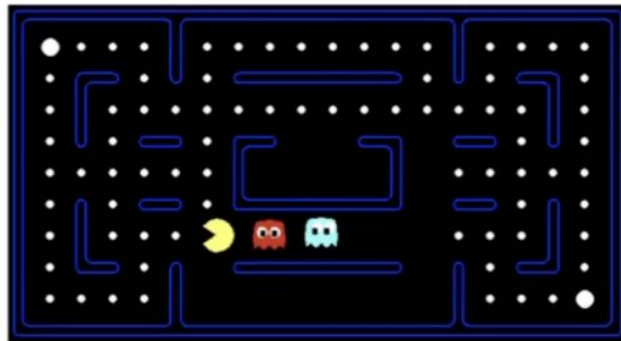
- Evaluation functions score non-terminal states
- Ideal function: returns the actual minimax value of the position
- In practice: typically weighted linear sum of features.

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- e.g., $f_1(s) = (\text{num white queens} - \text{num black queens})$
- Or a more complex nonlinear function (e.g., NN)



Evaluation of Pacman



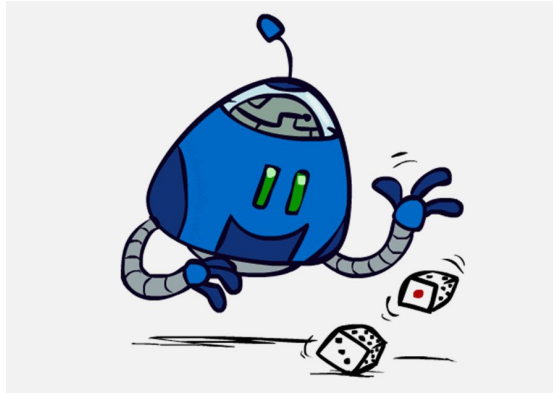
Evaluation Function and Alpha-Beta Pruning

- Amount of Alpha-Beta pruning depends on expansion ordering
- Evaluation function can provide guidance to expand most promising nodes first
- IF evaluation function provides upper-bound on value at min-node, and upper-bound is already lower than better option for max along path to root, then we can prune



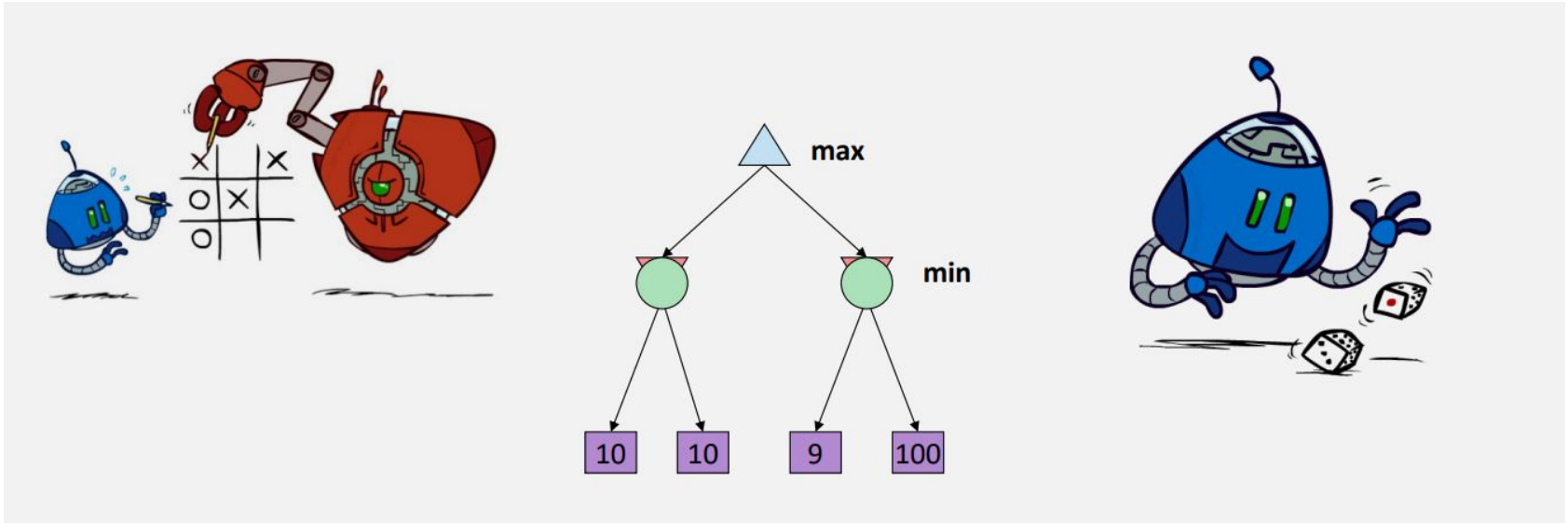
Uncertain Outcomes

- Source of Uncertainty:
 - Explicit randomness: rolling dice
 - Unpredictable opponents: the ghosts respond randomly
 - Actions can fail: when moving a robot, wheels might slip



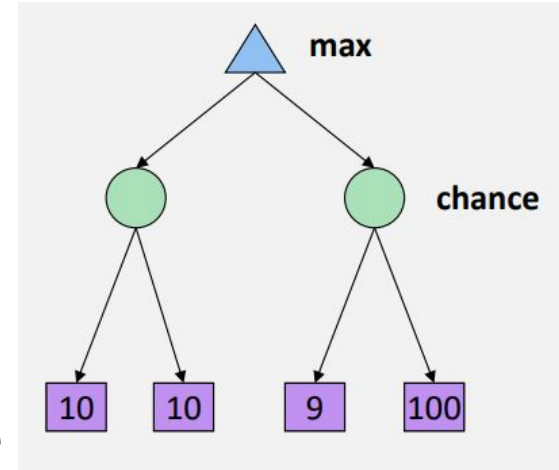
Worst-Case vs. Average Case

- Idea: Uncertain outcomes controlled by chance, not an adversary!



Expectimax Search

- Values should now reflect average-case (expectimax) outcomes
- not worst-case (minimax) outcomes
- Expectimax search: compute the average score under optimal play
 - MAX nodes as in minimax search
 - Chance nodes are like MIN nodes but the outcome is uncertain
 - Calculate their expected utilities



Expectimax Pseudocode

```
def value(state):
```

if the state is a terminal state: return the state's utility

if the next agent is MAX: return max-value(state)

if the next agent is EXP: return exp-value(state)

```
def max-value(state):
```

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

```
def exp-value(state):
```

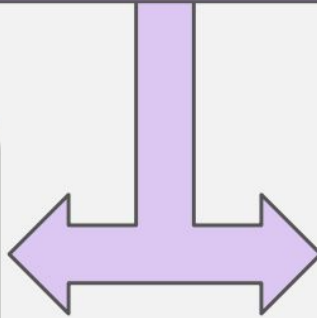
initialize $v = 0$

for each successor of state:

$p = \text{probability}(\text{successor})$

$v += p * \text{value}(\text{successor})$

return v



Expectimax Pseudocode

```
def exp-value(state):
```

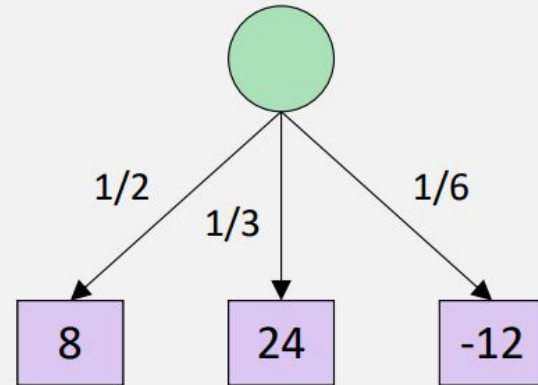
```
    initialize v = 0
```

```
    for each successor of state:
```

```
        p = probability(successor)
```

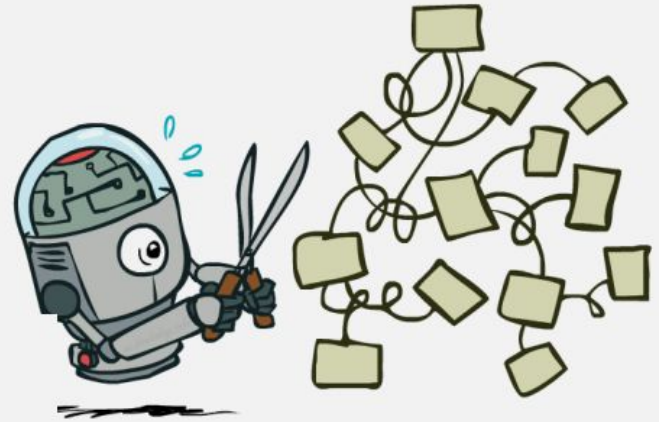
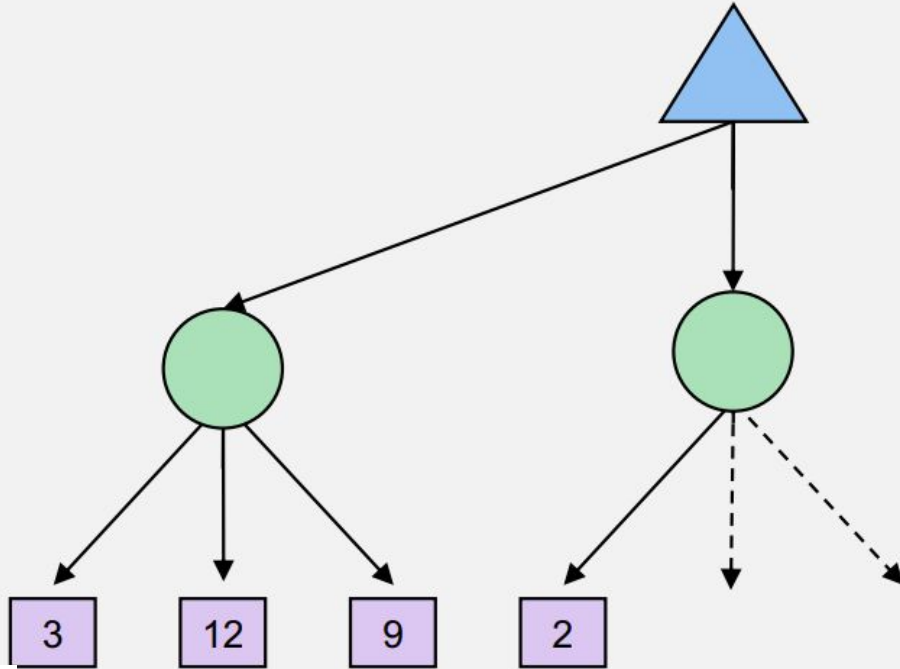
```
        v += p * value(successor)
```

```
    return v
```



$$v = (1/2) (8) + (1/3) (24) + (1/6) (-12) = 10$$

Expectimax Pruning?



The Dangers of Optimism and Pessimism

Dangerous Optimism

Assuming chance when the world is adversarial

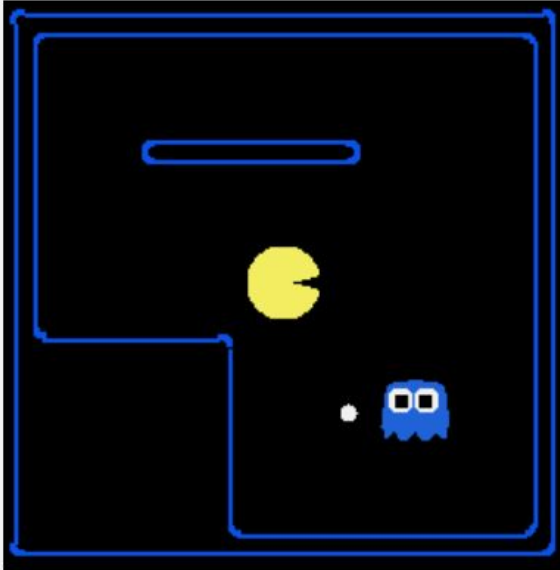


Dangerous Pessimism

Assuming the worst case when it's not likely



Assumptions vs. Reality



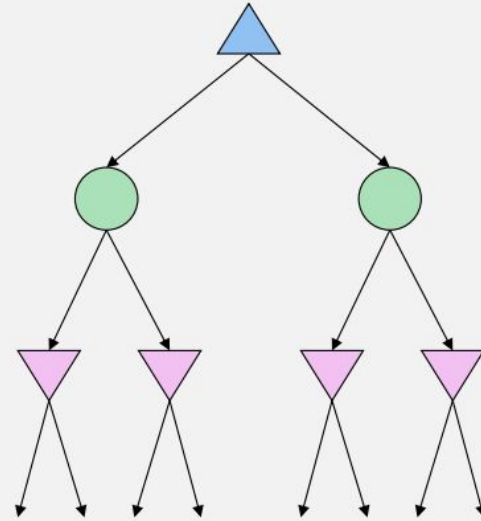
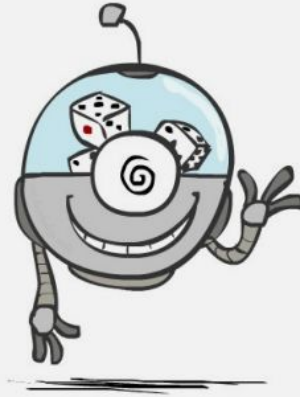
	Adversarial Ghost	Random Ghost
Minimax Pacman	Won 5/5 Avg. Score: 483	Won 5/5 Avg. Score: 493
Expectimax Pacman	Won 1/5 Avg. Score: -303	Won 5/5 Avg. Score: 503

Results from playing 5 games



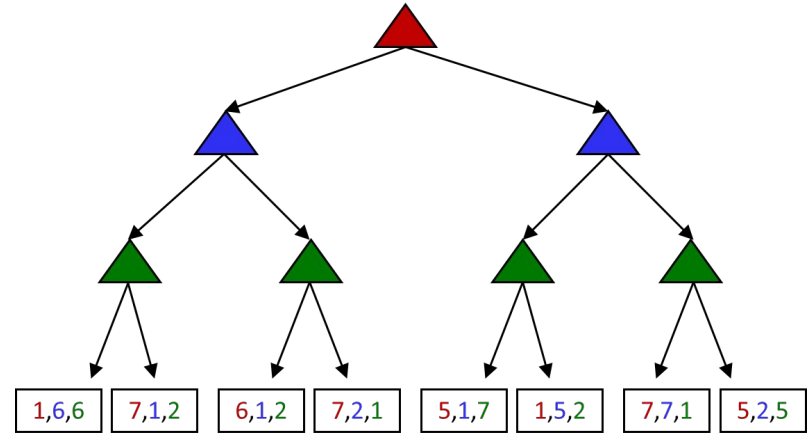
Mixed Layer Types

- E.g. Backgammon
- Expectiminimax
 - Environment is an extra “random agent” player that moves after each min/max agent
 - Each node computes the appropriate combination of its children



Multi-Agent Utilities

- What if the game is not zero-sum, or has multiple players?
- Generalization of minimax:
 - Terminals have utility tuples
 - Node values are also utility tuples
 - Each player maximizes its own component
 - Can give rise to cooperation and Competition dynamically ...



Next Session

- Probability & Uncertainty

